

GVP-MAAA Full Technical Report

1. Executive Summary

GVP-MAAA is a full-stack Academic Monitoring, Analytics, Alerts, and Assistance platform. It serves three primary user roles:

- Admin
- Faculty (Teacher)
- Student

The system combines operational academic modules (attendance, marks, assignments, events, resources, timetable), intelligence modules (risk scoring, alerts, placement analytics), and behavioral modules (SMARTask generation with XP, streak, and leaderboard).

This report is written as a single, GPT-friendly reference document that explains the project from initial structure to current implementation state.

2. Project Goals

2.1 Academic Operations

- Manage and monitor student attendance, marks, assignments, resources, and events.
- Support faculty workflows for entering attendance, uploading marks, and managing classroom content.
- Support admin workflows for governance and institutional monitoring.

2.2 Early Risk Detection

- Identify at-risk students using attendance, marks, and CGPA signals.
- Generate actionable alerts for students and faculty.

2.3 Placement Readiness and Tracking

- Track placement drives, eligibility, student applications, round progression, and outcomes.
- Provide placement intelligence and intervention insights.

2.4 Student Behavior Activation

- Generate smart tasks from real student data.
- Encourage daily consistency through streaks, XP, and motivational feedback.

3. Tech Stack

3.1 Frontend

- React (Vite)
- React Router
- Axios
- Tailwind CSS
- MUI Icons / MUI packages
- Recharts
- Framer Motion + Lottie (available in dependencies)

Frontend entry/config files:

- gvp-maaa/package.json
- gvp-maaa/src/main.jsx
- gvp-maaa/src/App.jsx
- gvp-maaa/vite.config.js
- gvp-maaa/src/index.css

3.2 Backend

- FastAPI
- Uvicorn
- SQLAlchemy ORM

- PostgreSQL (psycopg2)
- JWT auth (python-jose)
- APScheduler
- Pandas / NumPy / OpenPyXL
- Report generation tools (reportlab, python-docx)

Backend core files:

- gvp-maaa/Backend/main.py
- gvp-maaa/Backend/models.py
- gvp-maaa/Backend/schemas.py
- gvp-maaa/Backend/database.py
- gvp-maaa/Backend/auth.py
- gvp-maaa/Backend/requirements.txt

4. System Architecture

4.1 Architecture Style

- Client-server architecture
- SPA frontend + REST backend
- Monolithic backend application file with service/ML helper modules

4.2 Request Flow

1. User signs in from role-specific login pages.
2. Backend returns JWT access token.
3. Token is stored in localStorage.
4. Axios interceptors add Bearer token to each request.
5. Protected frontend routes enforce role access.
6. Backend processes business logic and database operations.
7. UI dashboards display real-time role-specific data.

4.3 Important Runtime Components

- Frontend router and role gating: App.jsx + ProtectedRoute.jsx
- API clients: src/utils/api.js and src/utils/axios.js
- Backend route host: Backend/main.py
- Database session dependency: Backend/database.py

5. Frontend Structure

5.1 Main Frontend Tree

- gvp-maaa/src/main.jsx
- gvp-maaa/src/App.jsx
- gvp-maaa/src/components/
- gvp-maaa/src/dashboards/
- gvp-maaa/src/pages/Admin/
- gvp-maaa/src/pages/Teacher/
- gvp-maaa/src/pages/Student/
- gvp-maaa/src/utils/
- gvp-maaa/src/styles/

5.2 Routing and Role Separation

- Public/auth routes: role selection + sign in/up + password reset
- Protected routes:
 - /admin (admin only)
 - /teacher/* (faculty only)
 - /student/* (student only)

5.3 Dashboard Shells

- Student dashboard:
 - Sidebar navigation
 - Profile panel
 - Alerts indicator
 - Page-level data fetching and rendering
- Teacher dashboard:
 - Academic workflows + quick actions
 - Placement and placement coordinator views
 - Alerts and profile management
- Admin dashboard:
 - Nested routes for overview, management, insights, settings, placement

5.4 UI System

- Tailwind utility classes + custom CSS animation library in index.css.
- Glassmorphism helpers and visual feedback patterns.
- Charting and metrics visualization using Recharts.

5.5 SPA Refresh Reliability

Vite config includes SPA fallback middleware so route refreshes (admin/student/teacher deep paths) are rved correctly by index.html instead of returning 404.

6. SMART Task Subsystem

The SMART task subsystem is a major product layer documented in:

- SMART_TASK_SYSTEM_DOCUMENTATION.md
- SMART_TASK_IMPLEMENTATION_GUIDE.md
- SMART_TASK_STUDENT_DATA_EXAMPLES.md
- SMART_TASK_SYSTEM_SUMMARY.md
- SMART_TASK_QUICK_REFERENCE.md

6.1 Core Files

- gvp-maaa/src/components/SmartTaskManager.jsx
- gvp-maaa/src/components/SmartTaskManager.css
- gvp-maaa/src/utils/taskGenerationEngine.js

6.2 Behavior

- Generates tasks from:
 - attendance
 - marks
 - pending assignments
 - upcoming events
 - study recovery logic
- Prioritization model: HIGH > MEDIUM > LOW
- Time buckets:
 - today (max 2)
 - this week (max 3)

6.3 Engagement Layer

- XP tracking
- Streak tracking
- Class leaderboard
- Task verification status (verified/pending)
- Predictive feedback text based on completion compliance

6.4 Backend Endpoints Supporting SMART Tasks

- /student/tasks/today
- /student/tasks/complete/{task_id}
- /student/xp/{student_id}

- /student/streak/{student_id}
- /class/leaderboard/{class_id}

7. Backend Structure

7.1 Backend Main Modules

- Backend/main.py: route orchestration and integration logic
- Backend/models.py: SQLAlchemy models
- Backend/schemas.py: Pydantic request/response schemas
- Backend/database.py: engine, session, Base
- Backend/auth.py: JWT and auth middleware helper
- Backend/security.py: password hashing/verification

7.2 Service Layer

- Backend/services/risk_engine.py
- Backend/services/alert_rules.py
- Backend/services/placement_engine.py
- Backend/services/admin_insights_engine.py

7.3 ML/Analytics Utility Layer

- Backend/ml/risk_engine.py
- Backend/ml/prediction_engine.py
- Backend/ml/recommendation_engine.py
- Backend/ml/insights_engine.py
- Backend/ml/alert_engine.py

7.4 Operational Utilities and Migration Scripts

Backend contains helper scripts for migrations, schema adjustments, verification, and testing:

- migrate_db.py, migrate_events.py, migrate_db_psycopg2.py, etc.
- fix_* scripts and verification scripts
- test_api.py, test_endpoints.py

8. Database Model Overview

Database is PostgreSQL-backed through SQLAlchemy.

8.1 Identity and Organization

- users
- departments
- students
- faculty

8.2 Academic Core

- subjects
- faculty_subject mapping
- attendance
- marks
- timetable
- assignments
- assignment_submissions
- resources
- resource_access
- events + event attendance/registration

8.3 Alerts and Settings

- alerts
- alert recipients

- warning and system alert entities
- system settings + settings audit logs

8.4 Placement Domain

- companies
- placement_drives
- student_drives
- drive_applications
- drive_faculty_map
- drive_coordinator_map
- placement_feedback
- audit logs and assignment history

8.5 Data Governance Patterns

- Soft-delete flags on key user entities.
- Department normalization support.
- Audit logging for critical updates.

9. API Surface (Domain-Wise)

The API surface is extensive and centralized in Backend/main.py.

9.1 Authentication

- login
- student/teacher signup
- admin login
- forgot/reset password

9.2 Student APIs

- profile
- attendance and monthly attendance
- marks and insights
- assignments and submission
- resources and access logs
- events and registration
- alerts
- placement + placement intelligence
- SMART task + XP + streak + leaderboard

9.3 Faculty APIs

- profile
- attendance creation and reporting
- marks upload/preview/validation/template
- assignment creation and submission review
- resource upload and tracking
- events + attendance + results
- alerts to students
- faculty overview and insights

9.4 Admin APIs

- overview and trend analytics
- student management and promotion
- teacher management
- subject management and assignment mapping
- timetable upload and lifecycle
- settings and settings logs
- alerts and interventions
- placement dashboard/analytics

- coordinator assignment management

9.5 Placement APIs

- company CRUD
- drive CRUD + lifecycle (open/close/reopen)
- eligibility and student-drive mapping
- faculty assignment to drives
- coordinator assignment/extension/revoke
- notify students (bulk/filter/individual)
- feedback and placement summaries

10. Security and Access Control

10.1 JWT

- Access token generation and verification in Backend/auth.py.
- Frontend interceptors attach Authorization headers.

10.2 Route Guards

- Frontend ProtectedRoute checks token and role.
- Backend endpoints use current user context and role checks.

10.3 Password and Recovery

- Hashed password flow through security helper.
- Forgot/reset flow includes token issuance and verification.

11. File and Upload Infrastructure

Backend mounts uploads directory for static serving.

Uploads include categorized folders:

- uploads/alerts
- uploads/assignments
- uploads/external_events
- uploads/resources
- uploads/timetables

This enables direct file access URLs and persistent media/resource attachment workflows.

12. Admin Intelligence Layer

Admin intelligence is a consolidated analytics capability.

12.1 Overview Metrics

- at-risk counts
- attendance risk percentage
- active alerts/events
- total students/teachers
- data completeness indicators

12.2 Insight Drivers

- risk summaries by department
- subject-wise failure concentration
- trend comparison with previous windows
- actionable outputs for interventions

Core implementation:

- Backend/services/admin_insights_engine.py

13. Risk and Alerting Logic

13.1 Risk Signals

- Attendance threshold checks
- Mid marks checks
- CGPA checks
- performance trend impacts

13.2 Alert Rules

- attendance alert
- marks alert
- CGPA alert
- no-data safety behavior to prevent false alerts

Core implementation:

- Backend/services/risk_engine.py
- Backend/services/alert_rules.py
- Backend/ml/risk_engine.py

14. Placement Intelligence Layer

Placement is a deep subsystem with readiness and intervention intelligence.

14.1 Core Functions

- readiness scoring
- company eligibility
- skill gap analysis
- interview insights
- selection probability
- action planning

14.2 Operational Support

- drive lifecycle management
- student application tracking
- faculty/coordinator mapping
- feedback and audit trails

Core implementation:

- Backend/services/placement_engine.py

15. Development Evolution (From Scratch to Current)

15.1 Initial Foundation

- Vite React frontend setup.
- FastAPI backend setup.
- Role-based authentication and dashboard skeletons.

15.2 Academic Expansion

- attendance, marks, assignments, timetable, resources, events modules.
- faculty and admin operational workflows.

15.3 Analytics and Monitoring

- admin overview and trend insights.
- risk scoring and alerting modules.

15.4 Placement Expansion

- full placement workflow with drives, roles, notifications, and intelligence APIs.

15.5 SMART Task Expansion

- student behavioral engine with task generation, verification, streaks, XP, and leaderboard.
- full documentation set for design, implementation, and examples.

15.6 Hardening and Stabilization

- SPA refresh fallback routing.
- role/permission enhancements for placement coordination.
- normalized departments support.
- marks template and endpoint reliability fixes.

16. Current Strengths

- Broad institutional feature coverage.
- Strong role-based separation and UX paths.
- Practical placement operations with intelligence.
- Real-time actionable student engagement (SMART tasks).
- Combined operational + analytical architecture.

17. Current Challenges / Technical Debt

- Backend main.py is very large and can be split into routers/services for maintainability.
- Secret/config hardcoding should be fully environment-driven.
- More automated integration and regression testing would improve reliability.
- API versioning and modular OpenAPI grouping would help scaling.

18. Run and Local Development

Root-level quick run notes currently exist in README.md.

Typical local flow:

1. Activate backend virtual environment.
2. Start FastAPI server with uvicorn.
3. Start frontend Vite dev server.
4. Access backend docs via /docs.

Reference files:

- README.md
- gvp-maaa/README.md

19. Full File Structure Summary (High-Level)

19.1 Root

- README.md
- SMART_TASK_*.md docs
- gvp-maaa/ (main app)

19.2 Frontend (gvp-maaa)

- src/
 - App.jsx, main.jsx, index.css
 - components/
 - dashboards/
 - pages/Admin/
 - pages/Teacher/

- pages/Student/
- utils/
- public/
- scripts/

19.3 Backend (gvp-maaa/Backend)

- main.py
- models.py
- schemas.py
- database.py
- auth.py
- services/
- ml/
- migration and verification scripts
- uploads/

20. GPT-Focused Understanding Notes

For any GPT model analyzing this project:

1. Treat this as a role-based multi-domain academic platform.
2. Main complexity lives in backend route orchestration and placement modules.
3. SMART task subsystem is both UX and behavior-engine logic.
4. Data model is relational and normalized around users, academics, alerts, and placement.
5. Frontend is page-driven with clear role partitions and API-based rendering.

21. Conclusion

GVP-MAAA has evolved from a standard role-based academic portal into a comprehensive institutional intelligence platform.

It now includes:

- full academic operations,
- risk and alert intelligence,
- placement readiness ecosystem,
- and student behavior activation through smart tasks.

This document is intended as the single-source, machine-readable and human-readable report for technical understanding, onboarding, and future scaling decisions.